

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода классификации данных с
использованием природных временных рядов.

Студент гр. 3384

Козьмин Н.В.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов

Задание.

Вариант 8.

Наименование станции: Тикси (ТХВУ). Период: 20-29 сентября 2015

1. Подготовить данные. Скачать данные из источника [<https://www.nmdb.eu/nest/>] за периоды и станции, указанные по вариантам таблиц. Использовать данные с одноминутным временным разрешением. При наличии пропусков в данных заполнить их медианным значением.
2. Сформировать векторы-историй. Временной ряд разбить на последовательные блоки (векторы) длиной m , каждый из которых должен содержать m прошлых значений. Последний вектор в ряде будет «тестовой» точкой для поиска соседей. Величину m выбрать равной 180 отсчетов.
3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей (k) определить равным 2-4.
4. Использовать соседей для прогноза. На основе найденных ближайших соседей построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков, нужно взять простое среднее следующих за ними значений.
5. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Для выполнения лабораторной работы был разработан алгоритм прогнозирования одноминутного временного ряда нейтронного монитора методом ближайших соседей. Сначала данные были загружены из NMDB NEST и приведены к полной минутной временной сетке. Пропущенные значения заполнены медианой доступных наблюдений. Далее сформированы векторы-истории длиной 180 отсчетов, для последнего вектора найдены ближайшие исторические аналоги по трем метрикам, а прогноз получен усреднением следующих за соседями фрагментов.

Загрузка и подготовка данных

Данные станции Тикси (TXBY) были загружены из NMDB NEST за период с 20.09.2015 00:00 UTC по 29.09.2015 23:59 UTC. Параметр `tresolution=0` задает минутное разрешение, а `dtype=corr_for_efficiency` соответствует показателю `RCORR_E`. Из ASCII-ответа были извлечены временные метки и значения показателя, после чего записи были преобразованы к типам `datetime (UTC)` и `float`.

На полной минутной сетке содержится 14400 позиций. Из NEST получено 14032 наблюдений; обнаружено 368 пропусков (2.556%). Пропуски заполнены медианой доступных наблюдений, равной 86.133. После заполнения в ряде осталось 0 пропусков.

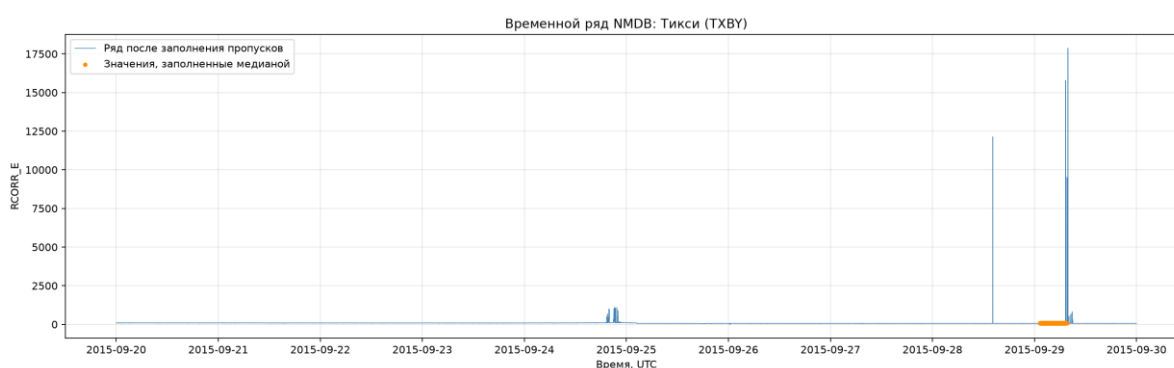


Рисунок 1 - Временной ряд TXBY после заполнения пропусков медианой

Формирование векторов-историй

Временной ряд разбит на скользящие векторы-истории длиной $m=180$ минут. Последние 180 значений ряда образуют тестовый вектор. Для каждого исторического вектора дополнительно сохранен следующий за ним фрагмент

длиной 180 минут, который используется как целевой фрагмент прогноза. Исторические векторы и их следующие фрагменты заканчиваются до начала тестового вектора, что исключает попадание тестовых данных в выборку соседей.

Получено 13861 исторических векторов размерности 180. Тестовый вектор также содержит 180 значений, а каждый целевой фрагмент - 180 значений.

Поиск ближайших соседей

Расстояние между тестовым вектором и каждым историческим вектором было рассчитано по евклидовой, манхэттенской метрикам и метрике Чебышёва. Для каждой метрики выбраны $k=4$ вектора с наименьшими расстояниями. Результаты поиска приведены в таблицах 1-3.

Таблица 1 - Ближайшие соседи: Евклидова метрика

Ранг	Расстояние	Начало истории	Конец истории
1	20.951	27.09.2015 22:05	28.09.2015 01:04
2	21.257	28.09.2015 04:57	28.09.2015 07:56
3	21.442	29.09.2015 13:53	29.09.2015 16:52
4	21.463	27.09.2015 22:46	28.09.2015 01:45

Таблица 2 - Ближайшие соседи: Манхэттенская метрика

Ранг	Расстояние	Начало истории	Конец истории
1	219.465	26.09.2015 20:34	26.09.2015 23:33
2	225.068	27.09.2015 13:28	27.09.2015 16:27
3	225.597	27.09.2015 22:05	28.09.2015 01:04
4	225.987	29.09.2015 13:53	29.09.2015 16:52

Таблица 3 - Ближайшие соседи: Чебышёва метрика

Ранг	Расстояние	Начало истории	Конец истории
1	3.600	26.09.2015 01:12	26.09.2015 04:11
2	3.733	28.09.2015 02:59	28.09.2015 05:58
3	3.866	27.09.2015 00:07	27.09.2015 03:06
4	3.867	26.09.2015 07:06	26.09.2015 10:05

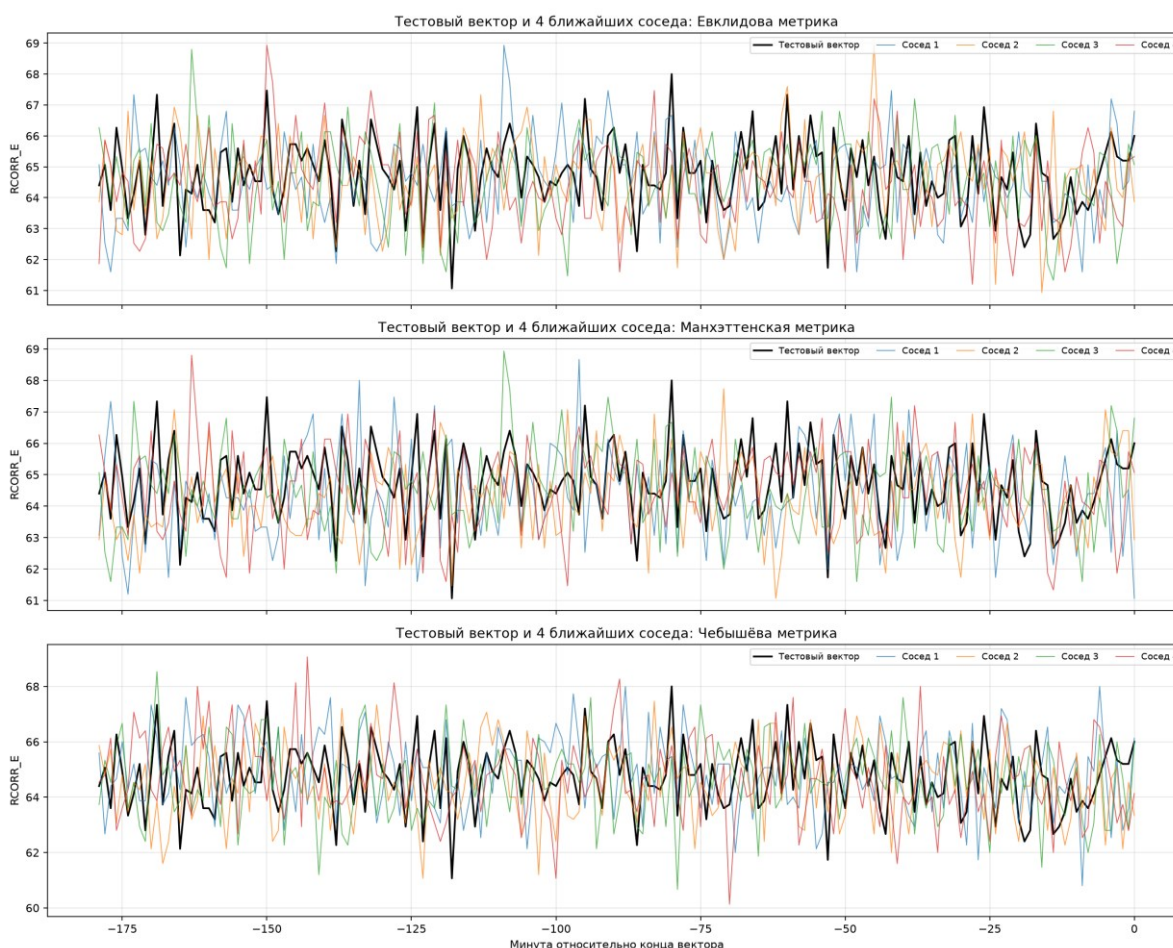


Рисунок 2 - Тестовый вектор и ближайшие исторические соседи

Прогноз следующих значений

Для каждой метрики прогноз на 180 минут сформирован как простое покомпонентное среднее четырех фрагментов, следующих за найденными соседями. Сводные характеристики прогнозов приведены в таблице 4.

Таблица 4 - Сводные характеристики прогнозов

Метрика	Мин. расстояние	Среднее	Минимум	Максимум
Евклидова	20.951	64.531	62.600	66.133
Манхэттенская	219.465	64.421	62.367	66.500
Чебышёва	3.600	64.737	63.267	66.666

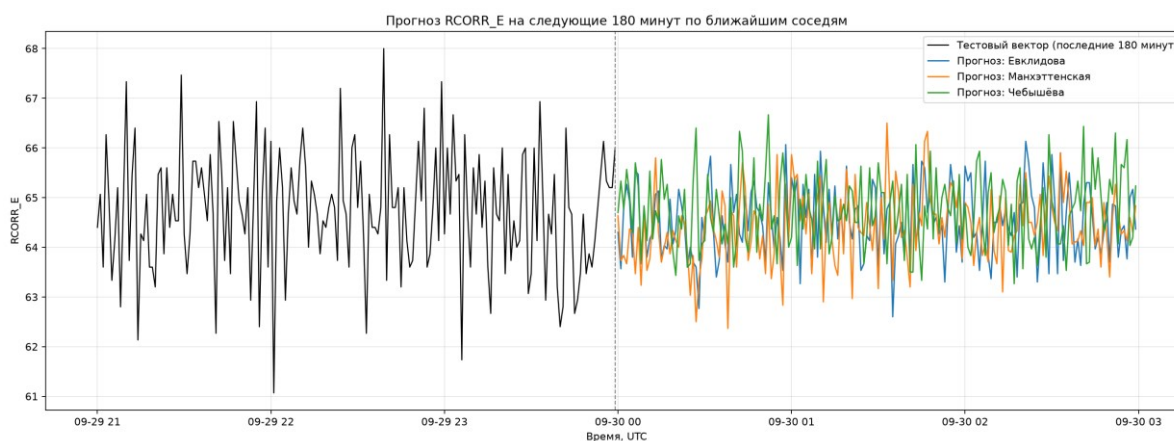


Рисунок 3 - Прогноз на 180 минут по трем метрикам

Первые десять точек каждого прогноза приведены в таблице 5.

Таблица 5 - Начальные значения построенных прогнозов

Время, UTC	Евклидова	Манхэттенская	Чебышёва
30.09.2015 00:00	64.300	64.633	64.700
30.09.2015 00:01	63.567	63.733	65.333
30.09.2015 00:02	64.833	63.833	64.767
30.09.2015 00:03	65.267	63.667	65.567
30.09.2015 00:04	64.933	64.367	65.066
30.09.2015 00:05	63.800	64.300	64.300
30.09.2015 00:06	65.534	63.467	65.700
30.09.2015 00:07	65.466	64.400	65.233
30.09.2015 00:08	63.666	63.233	63.733
30.09.2015 00:09	64.666	64.733	64.833

Выводы.

В ходе лабораторной работы построен прогноз временного ряда нейтронного монитора станции Тикси методом ближайших соседей. Данные за десять суток приведены к непрерывной минутной сетке, причем медианная замена применялась только к пропускам. Для последнего 180-минутного фрагмента найдены четыре наиболее близких исторических аналога по каждой из трех метрик. Усреднение следующих за этими аналогами фрагментов позволило получить три прогноза на 180 минут. Различия между прогнозами отражают различия в критериях близости временных векторов. Цель работы достигнута: реализован алгоритм поиска соседей и прогнозирования по природному временному ряду.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.ipynb

```
from io import StringIO
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests

try:
    from IPython.display import display
except ImportError:
    display = print

ROOT = Path.cwd()
cache_path = ROOT / "txby_2015-09-20_29_ascii.txt"
url = (

"https://www.nmdb.eu/nest/draw_graph.php?formchk=1&stations[]=TXBY"

"&tabchoice=revori&dtype=corr_for_efficiency&tresolution=0&yunits=0"
    "&date_choice=bydate&start_day=20&start_month=9&start_year=2015"
    "&start_hour=0&start_min=0&end_day=29&end_month=9&end_year=2015"
    "&end_hour=23&end_min=59&output=ascii"
)

if cache_path.exists():
    response_text = cache_path.read_text(encoding="utf-8")
else:
    response = requests.get(url, timeout=60)
    response.raise_for_status()
    response_text = response.text
    cache_path.write_text(response_text, encoding="utf-8")
data_lines = [
    line.strip()
    for line in response_text.splitlines()
    if line[:4].isdigit() and ";" in line
]
df = pd.read_csv(
    StringIO("\n".join(data_lines)),
    sep=";",
    names=["datetime", "corr_for_efficiency"],
)
df["datetime"] = pd.to_datetime(df["datetime"], utc=True)
df["corr_for_efficiency"] = pd.to_numeric(
    df["corr_for_efficiency"], errors="coerce"
)
df = (
    df.dropna()
    .sort_values("datetime")
    .drop_duplicates("datetime")
    .reset_index(drop=True)
)
```

```

print(f"Загружено корректных наблюдений: {len(df)}")
df.head(10)

start = pd.Timestamp("2015-09-20 00:00:00", tz="UTC")
end = pd.Timestamp("2015-09-29 23:59:00", tz="UTC")
full_index = pd.date_range(start, end, freq="min")

series =
df.set_index("datetime")["corr_for_efficiency"].reindex(full_index)
series.index.name = "datetime"
missing_mask = series.isna()
median_value = series.dropna().median()
filled_series = series.fillna(median_value)

assert filled_series.notna().all(), "После заполнения не должно
остаться пропусков"

data_summary = pd.DataFrame(
    {
        "Показатель": [
            "Полный минутный ряд",
            "Полученные наблюдения",
            "Пропуски",
            "Заполнено медианой",
            "Медиана",
        ],
        "Количество / значение": [
            len(full_index),
            int(series.notna().sum()),
            int(missing_mask.sum()),
            int(missing_mask.sum()),
            median_value,
        ],
    }
)
display(data_summary.style.format({"Количество / значение":
"{:.3f}"}))
print(f"Пропусков после заполнения: {filled_series.isna().sum()}")

fig, ax = plt.subplots(figsize=(16, 5))
ax.plot(filled_series.index, filled_series, linewidth=0.65,
color="steelblue", label="Ряд после заполнения пропусков")
ax.scatter(
    series.index[missing_mask],
    filled_series[missing_mask],
    s=10,
    color="darkorange",
    label="Значения, заполненные медианой",
    zorder=3,
)
ax.set_title("Временной ряд NMDB: Тикси (TXBY)")
ax.set_xlabel("Время, UTC")
ax.set_ylabel("RCORR_E")
ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig(ROOT / "series_with_filled_gaps.png", dpi=160,
bbox_inches="tight")

```

```

plt.show()
plt.close(fig)

m = 180
k = 4
forecast_horizon = 180
values = filled_series.to_numpy(dtype=float)
n_values = len(values)

test_start = n_values - m
test_vector = values[test_start:]
candidate_starts = np.arange(test_start - m - forecast_horizon + 1)

if len(candidate_starts) < k:
    raise ValueError("Недостаточно исторических векторов для поиска соседей")

history_vectors = np.stack([values[i : i + m] for i in
candidate_starts])
future_vectors = np.stack(
    [values[i + m : i + m + forecast_horizon] for i in
candidate_starts]
)

assert test_vector.shape == (m,)
assert history_vectors.shape[1] == m
assert future_vectors.shape[1] == forecast_horizon
assert candidate_starts[-1] + m + forecast_horizon <= test_start

print(f"Число исторических векторов: {len(history_vectors)}")
print(f"Размер тестового вектора: {test_vector.shape[0]}")
print(f"Горизонт прогноза: {forecast_horizon} минут")
print(f"Тестовый интервал: {full_index[test_start]} - {full_index[-1]}")

differences = history_vectors - test_vector
distances = {
    "Евклидова": np.sqrt(np.sum(differences ** 2, axis=1)),
    "Манхэттенская": np.sum(np.abs(differences), axis=1),
    "Чебышёва": np.max(np.abs(differences), axis=1),
}

neighbor_indices = {}
neighbor_tables = {}
predictions = {}
summary_rows = []

for metric_name, metric_distances in distances.items():
    selected = np.argsort(metric_distances, kind="stable")[:k]
    neighbor_indices[metric_name] = selected
    predictions[metric_name] = future_vectors[selected].mean(axis=0)

starts = candidate_starts[selected]
table = pd.DataFrame(
    {
        "Метрика": metric_name,
        "Ранг": np.arange(1, k + 1),
        "Расстояние": metric_distances[selected],
    }
)

```

```

        "Начало истории": full_index[starts],
        "Конец истории": full_index[starts + m - 1],
        "Начало следующего фрагмента": full_index[starts + m],
        "Конец следующего фрагмента": full_index[starts + m +
forecast_horizon - 1],
    }
)
neighbor_tables[metric_name] = table
summary_rows.append(
    {
        "Метрика": metric_name,
        "Минимальное расстояние": metric_distances[selected[0]],
        "Среднее прогноза": predictions[metric_name].mean(),
        "Минимум прогноза": predictions[metric_name].min(),
        "Максимум прогноза": predictions[metric_name].max(),
    }
)

neighbors_summary = pd.concat(neighbor_tables.values(),
ignore_index=True)
forecast_summary = pd.DataFrame(summary_rows)
display(neighbors_summary.style.format({"Расстояние": "{:.3f}"}))
display(forecast_summary.style.format({
    "Минимальное расстояние": "{:.3f}",
    "Среднее прогноза": "{:.3f}",
    "Минимум прогноза": "{:.3f}",
    "Максимум прогноза": "{:.3f}",
}))

fig, axes = plt.subplots(3, 1, figsize=(15, 12), sharex=True)
relative_time = np.arange(-m + 1, 1)

for ax, (metric_name, selected) in zip(axes,
neighbor_indices.items()):
    ax.plot(relative_time, test_vector, color="black",
linewidth=1.6, label="Тестовый вектор")
    for rank, selected_index in enumerate(selected, start=1):
        ax.plot(
            relative_time,
            history_vectors[selected_index],
            linewidth=0.8,
            alpha=0.75,
            label=f"Сосед {rank}",
        )
    ax.set_title(f"Тестовый вектор и 4 ближайших соседа:
{metric_name} метрика")
    ax.set_ylabel("RCORR_E")
    ax.grid(True, alpha=0.3)
    ax.legend(ncol=5, fontsize=8)

axes[-1].set_xlabel("Минута относительно конца вектора")
fig.tight_layout()
fig.savefig(ROOT / "neighbor_profiles.png", dpi=160,
bbox_inches="tight")
plt.show()
plt.close(fig)

```

```

future_index = pd.date_range(
    full_index[-1] + pd.Timedelta(minutes=1),
    periods=forecast_horizon,
    freq="min",
)
prediction_table = pd.DataFrame(
    {
        "datetime": future_index,
        "Евклидова": predictions["Евклидова"],
        "Манхэттенская": predictions["Манхэттенская"],
        "Чебышёва": predictions["Чебышёва"],
    }
)
assert prediction_table.iloc[:, 1:].notna().all().all()

fig, ax = plt.subplots(figsize=(16, 6))
ax.plot(
    filled_series.index[-m:],
    filled_series.iloc[-m:],
    color="black",
    linewidth=1.1,
    label="Тестовый вектор (последние 180 минут)",
)
for metric_name, color in [
    ("Евклидова", "tab:blue"),
    ("Манхэттенская", "tab:orange"),
    ("Чебышёва", "tab:green"),
]:
    ax.plot(future_index, predictions[metric_name], color=color,
            linewidth=1.3, label=f"Прогноз: {metric_name}")

ax.axvline(full_index[-1], color="gray", linestyle="--",
            linewidth=1)
ax.set_title("Прогноз RCORR_E на следующие 180 минут по ближайшим соседям")
ax.set_xlabel("Время, UTC")
ax.set_ylabel("RCORR_E")
ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig(ROOT / "forecast_comparison.png", dpi=160,
            bbox_inches="tight")
plt.show()
plt.close(fig)

display(prediction_table.head(10).style.format({
    "Евклидова": "{:.3f}",
    "Манхэттенская": "{:.3f}",
    "Чебышёва": "{:.3f}",
}))

```